TECHNISCHE UNIVERSITÄT
CHEMNITZ

Multisensorial Systems

Sensor Data Fusion and Implementation of Kalman Filter Variations

Professorship of Communication Engineering

Lecturer:
Prof.Dr. Klaus Mößner
Dominik Raeck, M.Sc.

Introduction

Autonomous Vehicles (AVs) require an accurate perception of their surrounding environment to function correctly. While GPS sensors play an important role in providing positioning information, relying solely on GPS navigation can be problematic due to signal loss or degradation in areas with poor satellite visibility. Moreover, GPS accuracy is prone to errors from atmospheric conditions and signal multipath. To address this issue, autonomous vehicles use a combination of sensors, including cameras, lidar, and radar, in addition to GPS. Cameras provide additional information, such as infrastructure and road markings, and GPS and camera data can be fused to get a comprehensive image of the environment. Moreover, the Kalman Filter can be used to reduce the noise from measurements and estimate the state of the vehicles. By using the Kalman Filter in combination with other sensors, the traffic flow will be improved, and AVs can navigate their environment with more accuracy and safety.

Installation Python

To get started with Python, you will need to download and install it from the official Python download page, based on your operating system. Visit https://www.python.org/downloads/ and follow the installation instructions.

Editor for Python

You will need an editor or integrated development environment (IDE) to work with Python. Two popular and free options are Visual Studio Code (VS Code) and Atom. They are available on Windows, Mac, and Linux.

Installation libraries

As you work with Python, you will need to install libraries to extend its functionality.
In this course, we will use the NumPy, matplotlib, and itertools libraries. You can install these libraries using pip, a package manager for Python.

To install a library using pip, open a command prompt or terminal window. On Windows, press the “Windows key” + “R” and type “cmd” to open a command prompt. On Mac or Linux, search for “Terminal” in the application menu. Then, type the following command:

pip install “type the name of the library”

Tasks

Task 1: Reading the Dataset

The aim of this task is to read a dataset in Python and create measurements to use as input for the linear Kalman Filter and Extended Kalman Filter.

Subtask 1: Reading data

Read data from a CSV file in Python.

Note: There are multiple ways to read a CSV file in Python but remember that the shape of the result is a NumPy array and must be the same as Figure 1 (a list of tuples).

```
[(1, 1, 65.51258485, -18.82590766, -1.20922, -1.1046 , 225.19817)
 (1, 2, 65.44324765, -18.81187851, -1.24001, -1.14077, 225.18326)
 (1, 3, 65.30271932, -19.01106419, -1.27181, -1.17779, 225.16818)
 (1, 4, 65.31671228, -19.0910688 , -1.30459, -1.21548, 225.1524 )
 (1, 5, 65.23167278, -19.06586776, -1.33819, -1.25367, 225.13279)]
```

Fig.1: The first 5 rows of data

Subtask 2: Creating measurements

Create an array containing all measurements based on what you have learned in the lecture about the State space model. Depending on whether we are using a Linear KF or EKF, the way that these measurements are created can differ. Define a function that takes the data as input and generates the appropriate measurements.

There are multiple cars in each dataset, therefore data are grouped by their ID. Extract elements from the variable (groups), for this purpose, list comprehension is used. Complete the rest of the function and extract the first car (ego car) from the list that you created. Apply the same approach to Extended Kalman Filter. Consider that the measurement models for linear Kalman Filter and Extended Kalman Filter are different. The result of measurements as input must be the same as Figure 2 (A 2d NumPy array matrix with 4 rows and 1 column).

Note 1: You must convert the heading value from degree to radian for later usage.

Note 2: For grouping data by ID, itertools module is used which is a standard library in Python that provides a set of tools for working with iterable objects such as lists. These tools are designed to be fast, memory-efficient, and easy to use.

```
[[[ 65.51258485]
  [-18.82590766]
  [  1.63778942]
  [  3.93044954]]

 [[ 65.44324765]
  [-18.81187851]
  [  1.68492759]
  [  3.93018931]]]
```

Fig 2 (The first 2 arrays of measurements)

Task 2 & 3: Implementation of linear Kalman Filter and Extended Kalman Filter

The aim of these two tasks is to define the linear Kalman Filter and Extended Kalman Filter (EKF) algorithm.

Subtask 1: Implementation algorithms

The timestep (dt) has been given to you, define the variables and algorithm of the linear KF and EKF based on what you have learned in the lecture.

As a reminder, there are two steps for both Kalman Filter and Extended Kalman Filter

1. Prediction step
2. Update step

The variables u (control input vector) and B (control input matrix) are used in the state transition equation to model the dynamics of the system being measured. The control input matrix (B) is given to you. It maps the control input vector u to the state vector x. It defines how the control input affects the system dynamics. The control input vector (u) represents the external influence or driving force on the system. Velocity and heading value affect the dynamics of the system, therefore you must update the velocity and heading at each time step and calculate u depending on the state model.

Subtask 2: Filtering data with KF

Define a Function that takes the measurement array from Task 1 and uses the appropriate algorithm based on the user's choice (kf/ekf). The final shape must be the same as in Figure 2 (A 2d NumPy array with 4 rows and 1 column).

Note: Forecasting in task_2.py and task_3.py belong to Task 5.

Task 4: Visualization

The aim of this task is to visualize noisy measurements and the Filtered data by using KF or EKF and compare the results to each other.

Note: This task can be done after completing each task. For example, after completing task 1, you can visualize noisy measurements and see the result.

Subtask 1: Visualization of the results

Visualize measurements, KF, and EKF results.

Subtask 2: Visualization with background image

The aim of this subtask is to visualize the noisy data, KF, and EKF results. Add the background image related to the dataset that you are working with it. For this purpose, use Python's Matplotlib library along with the "imread" function from the "image" module. In addition, we need to convert the positions from meters to pixel values to align them with the background image. Use variable (s) to convert positions from meters to pixel values.

By incorporating the background image and converting the positions to pixel values, we can create a more informative visualization that allows us to better understand the performance of the KF and EKF filters on the noisy data.

Task 5: Forecasting

The aim of this task is to forecast the future trajectory of vehicles with Linear KF and EKF. The default values for the time step and the number of iterations for forecasting are f_dt = 0.2 and n=5, respectively. However, these values can be adjusted by the user.

Subtask 1: Forecasting with linear KF and EKF

Complete the existing forecast functions in task_2.py and task_3.py by defining a function that takes the timestep and measurement as input and returns forecasted points. The output must be a 2d NumPy array with 4 rows and 1 column, matching the shape of Figure 2. Then define a function that takes measurements, timestep, and the number of iterations (n) as inputs and uses a forecasting function appropriate for the chosen filter(kf/ekf). The output of this function must also be a 2d NumPy array with 4 rows and 1 column, matching the shape of Figure 2.

Subtask 2: Average Displacement Error (Bonus task)

This subtask aims to evaluate the performance of forecasting by Kalman Filter and EKF by the Average Displacement Error (ADE) method. ADE is a common metric used to evaluate the performance of trajectory prediction models. ADE measures the average distance between each forecasted point and the true measurement of an object in a given time interval.
Define a function that takes measurement and forecasted points as input and calculates ADE.